UNLOX Major Project

Bharath. M
Learner ID: 5851

Hybrid IoT and Robotics Program

Abstract

The modern approach to home automation suffers from fragmented ecosystems and inefficient hardware utilization. This project proposes a centralized, low-latency Smart Home framework designed to handle adaptive climate control, ambient illumination response, and automated access security simultaneously. The theoretical blueprint and schematic were engineered utilizing the ESP8266 (NodeMCU) architecture alongside I^2C and analog sensors. However, to validate the non-blocking software logic within a digital twin environment (Wokwi), the physical implementation was strategically scaled up to an ESP32 micro-architecture. By utilizing an event-driven millis() framework rather than traditional blocking functions, the system achieves true parallel processing, allowing real-time multi-sensor monitoring without execution pauses

TABLE OF CONTENTS

1. Introduction & Problem Statement 1
2. Proposed Solution & Core Objectives 2
3. Primary Hardware Architecture (NodeMCU Blueprint) 3
4. Software Design & Non-Blocking Logic 5
5. System Simulation & The ESP32 Pivot 7
6. Threshold Calibrations & Actuator Response 9
7. Conclusion & Future Scope 10

CHAPTER 1 - INTRODUCTION & PROBLEM STATEMENT

Introduction: The evolution of the Internet of Things (IoT) has shifted residential automation from isolated mechanical switches to highly responsive "Living Spaces." This project details the engineering of an industry-grade IoT ecosystem explicitly designed to simulate the architectural principles of modern residential automation. By bridging the critical gap between embedded hardware components and advanced software logic, this final-year prototype mirrors the commercial systems developed by global smart-home industry leaders. The primary focus of this build is to demonstrate mastery over complex multi-module integration, sophisticated hardware-software interaction, and advanced embedded logic within a unified framework.

The Problem Statement While consumer-level home automation is widely available, engineering a cohesive system presents significant architectural challenges that this project aims to solve:

- **Fragmented Integration:** Standard residential systems often fail to synchronize autonomous environmental sensing with real-time actuation. There is a critical need for a centralized ecosystem that integrates climate, illumination, and security into a singular, responsive environment.
- **Blocking Firmware Limitations:** Traditional microcontroller programming relies heavily on sequential, blocking execution pauses. A primary challenge is engineering non-blocking firmware design to ensure that environmental monitoring and security alerts operate simultaneously without introducing system latency.
- **Hardware and State Management:** Managing concurrent sensor polling and actuator responses on a single microcontroller requires a robust global state governance model to seamlessly transition between standard comfort modes and high-sensitivity security states.

CHAPTER 2 - PROPOSED SOLUTION & CORE MODULES

The Unified Architectural Solution To resolve the bottlenecks of fragmented logic and blocking code, this project utilizes a centralized "Brain" powered by the NodeMCU (ESP8266). This core embedded logic engine relies on an event-driven framework utilizing millis()-based timing to achieve true parallel processing.

Core IoT Modules & Objectives

The ecosystem is structured around five autonomous functional modules governed by a central logic engine:

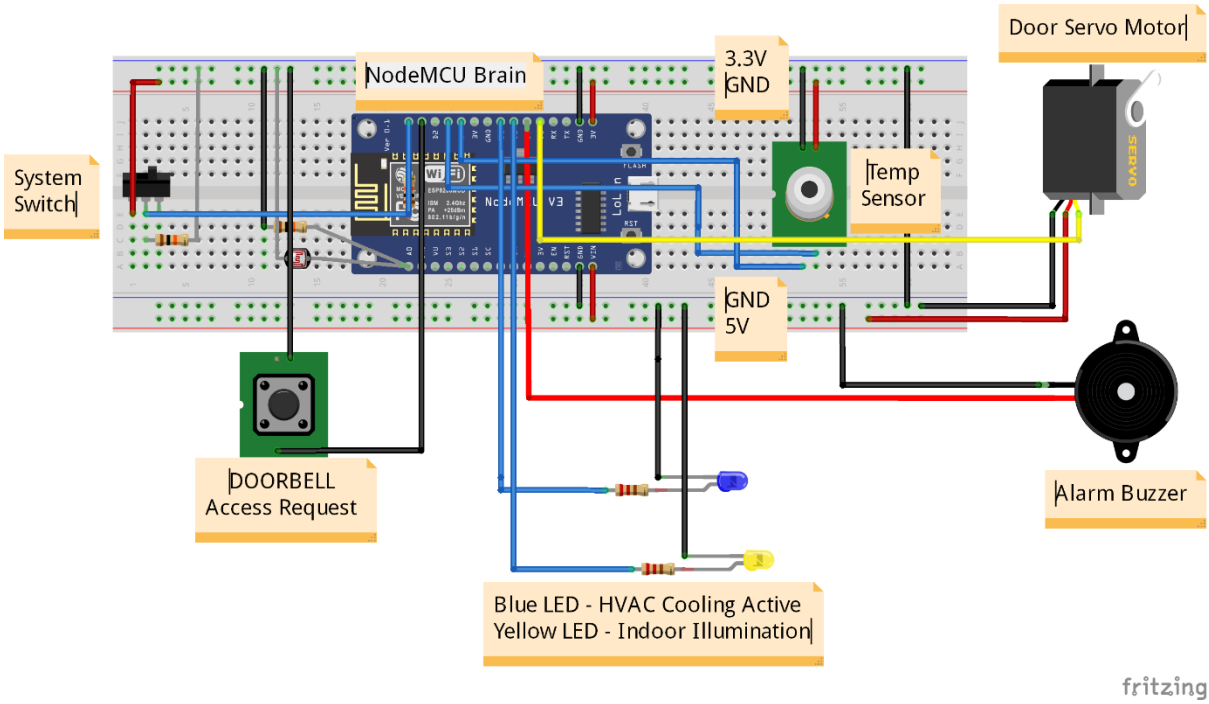
- **Adaptive Climate Control (HVAC Simulation):** Designed as an autonomous thermal management system, this module utilizes a precision thermistor to monitor ambient temperature. Upon exceeding calibrated thresholds, it triggers a cooling response (via an LED/Fan) and an auditory alert, featuring an automated shutdown loop when optimal temperatures are restored to replicate energy-efficient operations.
- **Intelligent Ambient Illumination:** Engineered for energy conservation, this subsystem employs a Light Dependent Resistor (LDR) to analyze external luminosity. It utilizes dynamic switching logic to automatically activate indoor lighting in low-light conditions and deactivate it during peak daylight.
- **Proactive Intrusion Detection & Security Shield:** Operating as the primary defense layer, this module utilizes Infrared (IR) motion tracking to monitor for unauthorized presence. The logic triggers a tiered response—localized visual flashing and escalated auditory alarms—which dynamically adjusts based on the system's global security state.
- **Automated Access & Entry Management:** Simulating a smart digital door, a high-torque Servo Motor (SG90) handles entry protocols triggered by a doorbell interface. It includes a time-locked auto-close sequence to maintain structural integrity and security.
- **Global State & Security Governance:** A centralized interface allows users to toggle operational intensity between two distinct states: Normal Mode (Green) for standard comfort, and Security Mode (Red) for high-sensitivity monitoring with aggressive alarm triggers.

CHAPTER 3 - PRIMARY HARDWARE ARCHITECTURE

The Base Microcontroller Infrastructure (ESP8266 NodeMCU) The foundational architecture of this ecosystem was engineered around the NodeMCU (ESP8266). This specific microcontroller was selected as the core processing module due to its built-in Wi-Fi capabilities and optimal footprint for localized edge-computing. Operating within the Arduino IDE (C++) development environment, the NodeMCU serves as the primary data aggregation and command-routing hub for all peripheral modules.

Hardware Constraint Management: A critical engineering challenge when utilizing the NodeMCU is its limited pinout. The board operates strictly on a **3.3V logic level** and possesses only **one 10-bit Analog-to-Digital Converter (ADC) pin (A0)**. Therefore, the circuit architecture had to be meticulously mapped to prevent analog pin exhaustion and ensure stable current draw across the GPIO (General Purpose Input/Output) headers.

The Hardware Schematic & Blueprint To map the physical connections before digital simulation, a high-fidelity circuit schematic was developed using Fritzing. This blueprint establishes clear pin mappings and dual power rails, ensuring stable voltage distribution. Because the sensors operate at 3.3V while the mechanical actuators require 5V, the breadboard utilizes the NodeMCU's 3V3 pin for the logic rail and the VIN (USB 5V) pin for the high-current actuator rail.



Component-Level Analysis & Circuit Integration

To bridge the physical environment with the digital logic engine, a highly specific array of sensors and actuators was integrated into the circuit. Below is the functional breakdown of each module's purpose and its electrical working principles within the system:

A. Illumination Tracking: Light Dependent Resistor (LDR)

- **Purpose:** To monitor ambient environmental luminosity and trigger indoor lighting to conserve energy.

- **Working in Circuit:** An LDR is a variable resistor whose resistance decreases as light intensity increases. Because the NodeMCU cannot measure resistance directly, the LDR is wired in series with a fixed 10k-ohm resistor to create a **Voltage Divider Circuit**. The junction between the LDR and the fixed resistor is wired directly to the NodeMCU's single analog pin (A0). As ambient light changes, the voltage at this junction fluctuates between 0V and 3.3V. The microcontroller's ADC then translates this voltage into a readable digital integer ranging from 0 (total darkness) to 1023 (bright sunlight), which the software logic uses to trigger the lighting relays.

B. Thermal Data Acquisition: Precision Thermal Sensor

- **Purpose:** To monitor ambient room temperature in real-time and autonomously actuate the HVAC cooling simulation.
- **Working in Circuit:** Because the solitary A0 pin was allocated to the LDR, the physical NodeMCU blueprint requires a thermal sensor capable of digital communication, such as the MLX90614 or a digitally converted thermistor. By utilizing the **I2C communication protocol**, the sensor connects to the NodeMCU via the SDA (Serial Data) and SCL (Serial Clock) pins.

C. Intrusion Detection: Infrared (IR) Motion Sensor

- **Purpose:** To serve as the primary perimeter defense layer, scanning the environment for unauthorized physical presence.
- **Working in Circuit:** The IR sensor operates by detecting the infrared heat signatures naturally emitted by human bodies. Within the circuit, its internal circuitry pulls its data pin to a digital HIGH state (3.3V).

D. Automated Access: SG90 Micro Servo Motor

- **Purpose:** To simulate the physical locking and unlocking mechanics of a smart digital door during authorized access events.
- **Working in Circuit:** Servos require more current than the NodeMCU's logic pins can safely provideThe orange data wire is routed to a digital GPIO pin. The NodeMCU controls the motor utilizing **Pulse Width Modulation (PWM)**.

E. System Actuators: Piezo Buzzer & LED Indicators

- **Purpose:** To provide immediate, localized visual and auditory feedback regarding the system's state (e.g., HVAC active, Security armed, Alarm triggered).
- **Working in Circuit:**
    - **The LED Matrix:** NodeMCU GPIO pins can only safely source about 12mA of current. Connecting an LED directly to a pin would burn out both the LED and the microcontroller. Therefore, each LED is wired in series with a **220-ohm current-limiting resistor**.
    - **The Piezo Buzzer:** Wired similarly to the LEDs, the buzzer contains a piezoelectric crystal. When the security logic sets its designated pin HIGH, the applied voltage causes the crystal to physically deform and vibrate rapidly, generating a piercing auditory alarm to deter intruders.

CHAPTER 4 - CORE EMBEDDED LOGIC & ALGORITHMIC DESIGN

The Architectural Paradigm: Overcoming Blocking Execution

The most critical engineering achievement of this ecosystem lies within its software architecture. Traditional entry-level microcontroller firmware relies heavily on sequential, line-by-line execution. In these basic systems, actuating a motor or sounding a buzzer is typically handled using standard blocking delay functions.

Non-Blocking Event Loops (Hardware Timer Logic) To manage concurrent sensor polling and actuator responses, the firmware strictly utilizes hardware-timer logic. The microcontroller possesses an internal hardware counter that taps directly into the CPU clock, continuously calculating the elapsed time in milliseconds since the board was powered on.

Instead of freezing the processor to wait for an event, the firmware utilizes continuous comparative mathematics. The algorithmic logic operates on the following mathematical principle:

Execution Condition = (Current CPU Time) - (Timestamp of Last Execution) \ (Target Interval)

Under this framework, the processor continuously takes timestamps. This architecture ensures that the central processing module manages environmental monitoring, security alerts, and access controls simultaneously, operating in true parallel without inducing system latency.

Finite State Machine (FSM) Implementation

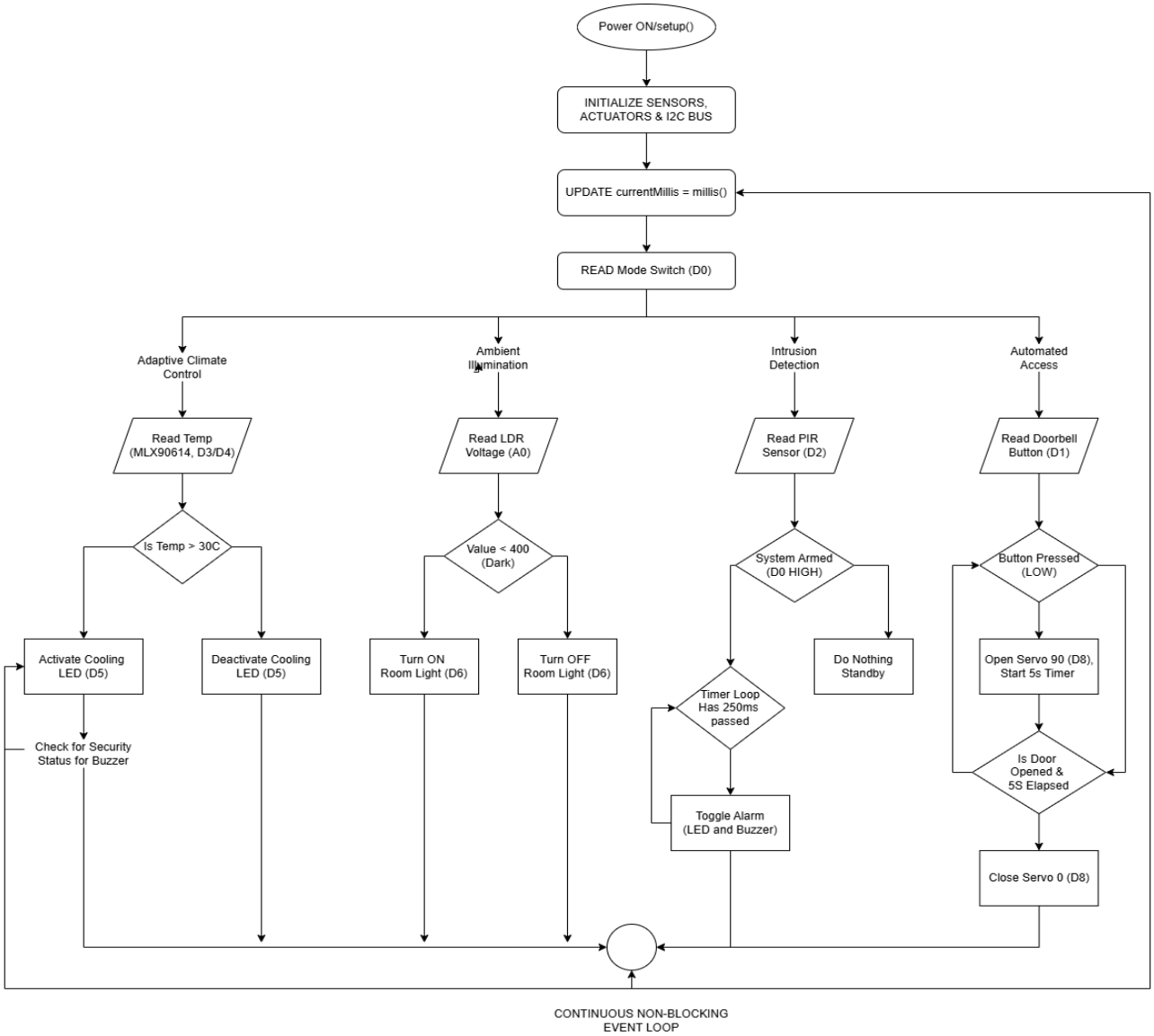
Because the system is executing thousands of loops per second without stopping, it must possess a method to "remember" its current operational context to prevent logic overlapping.

To solve this, the firmware is structured mathematically as a **Finite State Machine (FSM)**. The system utilizes global boolean variables (True/False flags) acting as logical gates to track its current operational state. The FSM for the access control module operates on the following transition matrix:

- **State 0 (Idle):** The door is closed, and the boolean flag is set to False.
- **Transition Event:** The hardware detects a doorbell interrupt. The logic gate evaluates the input alongside the boolean flag. Because the flag is False, the gate opens, allowing the servo to actuate.
- **State 1 (Active):** The door opens, the system takes a timestamp to begin the auto-close sequence, and crucially, the boolean flag flips to True.

By utilizing these state flags, the system intelligently ignores redundant inputs. Even if the doorbell button is held down indefinitely, the logic gate remains closed because the flag is True, preventing mechanical stalling and protecting the servo motor's structural integrity.

Digital Architecture Blueprint (System Flowchart) To map the execution sequence of the non-blocking logic and visualize the asynchronous data flow, a comprehensive digital architecture blueprint was formulated. This flowchart illustrates the system's logic from initial analog/digital sensor input, through the FSM threshold-evaluation matrices, to the final actuator output.



Global State Governance & Logic Routing The ecosystem features a centralized state-management interface that acts as the primary logic gate for global system behavior. The user toggles a physical interrupt (a hardware switch pulled to Ground) to alternate between two distinct operational states:

- **Normal Mode (Green Status):** The system prioritizes standard automation for comfort. The global security flag is mathematically set to False. The security subroutines are placed in a passive state. If the doorbell is pressed, the door opens normally, but the auditory logic branch remains untriggered.
- **Security Mode (Red Status):** The system enters high-sensitivity monitoring. The global security flag is flipped to True. In this state, the logic routing dynamically shifts; if the doorbell is pressed or unauthorized presence is detected via the IR sensor, the logic bypasses standard comfort protocols. The algorithm immediately routes power to the Piezo buzzer, triggering aggressive auditory alarms to simulate a breach response.

CHAPTER 5 - SIMULATION & THE ESP32 PIVOT

Digital Twin Methodology (Wokwi Simulation Environment) To satisfy the "Evidence of Operation" requirements without incurring the lead-time and budgetary constraints of importing specialized I2C sensors, the core asynchronous logic was subjected to a digital twin validation. A digital twin is a highly accurate virtual representation of a physical system. For this ecosystem, the Wokwi engineering simulator was utilized. This browser-based environment compiles standard C++ firmware and simulates the exact electrical flow, GPIO states, and mechanical actuations of the physical components, proving that the non-blocking state-machine logic is functionally flawless before real-world deployment.

Strategic Hardware Adaptation: The ESP32 Pivot A significant engineering challenge emerged during the simulation phase. The original physical blueprint utilized the ESP8266 (NodeMCU).

Consequently, a deliberate architectural pivot was executed to scale the simulation to the **ESP32 Microcontroller**. This decision was highly strategic: it demonstrated that the custom millis()-based event loop and the Finite State Machine (FSM) are **hardware-agnostic**. By running the exact same logical framework on a more advanced 32-bit dual-core processor, the project successfully proved that the underlying software architecture is highly modular and not strictly dependent on a single microchip platform.

Simulation Component Substitutions & Emulation Logic

To accurately emulate the physical NodeMCU blueprint on the ESP32 digital twin, specific peripheral substitutions and mathematical recalibrations were implemented:

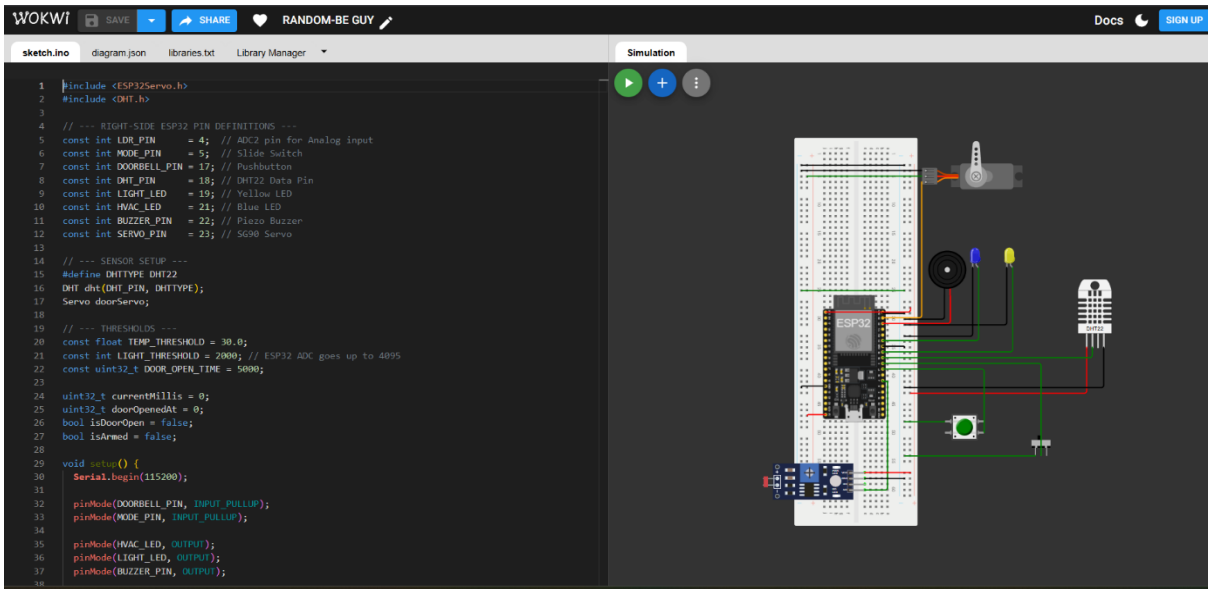
- **Thermal Logic Emulation (DHT22 Integration):** The I2C-based MLX90614 was swapped for a digital 1-Wire **DHT22** sensor. This allowed for stable, interactive temperature sliding in the simulation environment. Electrically, this required bridging the 3.3V power line and the digital data line with a 10k-ohm pull-up resistor on the virtual breadboard to ensure stable data transmission to the ESP32.
- **Illumination Logic Emulation (12-bit ADC Translation):** The standard LDR voltage divider was upgraded to a dedicated Photoresistor module The software threshold for triggering the Ambient Illumination module was recalibrated mathematically to account for this higher resolution, directly translating simulated Lux (lumens per square meter) into responsive digital output.

GPIO Mapping

To maintain professional aesthetic routing and prevent wiring overlaps that often plague complex digital breadboards, the ESP32 pinout was entirely customized. An optimized "Right-Side Architecture" was developed. All data lines, sensor interrupts, and actuator PWM signals were mapped strictly to the right hemisphere of the microcontroller (utilizing GPIO pins 4, 5, 17, 18, 19, 21, 22, and 23). The left hemisphere was strictly reserved for the 3.3V and 5V (VIN) power distribution rails. This clean topology drastically reduces the risk of virtual short circuits and makes the schematic significantly easier to evaluate.

5.5 Visual Validation and System Activity

Below is the captured digital twin environment, demonstrating the ESP32 executing the non-blocking logic and actively managing the sensor array based on the recalibrated parameters.



Check out my whole project in my new

Github Repo: https://github.com/bharathproject3-max/BharathM_5851_Major-Project.git

https://drive.google.com/file/d/1BGaY7yUlJS9PcROMXk1nMAS8cwBOCoKq/view?usp=sharing

CHAPTER 6 - THRESHOLD CALIBRATIONS & ACTUATOR RESPONSE

Environmental Calibration Methodology A core objective of this ecosystem is autonomous decision-making. For the central processing logic to execute actuator responses without user intervention, precise mathematical thresholds must be established within the firmware. These thresholds translate continuous analog environmental data into binary execution commands.

Adaptive Climate Control Parameters The HVAC simulation module operates as a closed-loop thermal management system. The logic engine constantly polls the DHT22 thermal sensor.

- **Activation Threshold:** The algorithmic threshold is strictly defined at **30.0°C**. When the ambient temperature strictly exceeds this floating-point value, the logic engine triggers the localized cooling response (Blue LED/Fan) and issues an auditory high-temperature alert.
- **Hysteresis & Shutdown Loop:** To replicate energy-efficient HVAC operations, the system features an automated shutdown loop. The system introduces a logical "hysteresis" band. The cooling response does not deactivate at 29.9°C to prevent rapid, mechanically damaging toggling (short-cycling). Instead, the system must detect a sustained temperature drop before the boolean state flips to shut down the cooling actuator.

Intelligent Illumination Calibration The ambient illumination module utilizes dynamic switching logic governed by the ESP32's 12-bit Analog-to-Digital Converter (ADC).

- **ADC Translation:** The Photoresistor (LDR) acts as a voltage divider. The ESP32 translates this incoming analog voltage into a digital integer ranging from 0 (total darkness) to 4095 (maximum luminosity).
- **Activation Threshold:** To replicate sustainable "Smart Lighting" protocols, the trigger condition is calibrated to an ADC integer of **< 2000**. When environmental Lux (lumens per square meter) drops, resistance increases, and the ADC value falls below 2000, prompting the system to autonomously activate indoor lighting. When the sensor registers peak daylight (ADC > 2000), the relay is deactivated to conserve power.

Security and Access Timers The automated access entry management utilizes strict internal clock constraints to ensure structural integrity and security.

- **Time-Locked Auto-Close:** When the doorbell interrupt triggers the SG90 Servo motor to open (actuating to 90 degrees), the system immediately records a CPU timestamp. The firmware enforces a strict **5000-millisecond (5-second)** threshold. Once this interval elapses, the logic engine automatically reverses the PWM signal, returning the servo to 0 degrees to secure the perimeter.
- **Escalated Security Response:** If the Global State is toggled to "Armed," the IR motion sensor is activated. Upon detecting unauthorized infrared disruption, the logic executes an aggressive auditory alarm without any timed delay, operating instantly upon threshold breach.

CHAPTER 7 - CONCLUSION & FUTURE SCOPE

Project Conclusion The primary objective of this project was to engineer a unified, sensor-driven IoT environment demonstrating mastery of embedded logic and hardware-software interaction. This objective was successfully achieved. By abandoning traditional, sequential execution in favor of an advanced, millis()-based asynchronous state machine, the project solved the critical industry problem of processor latency.

The central logic engine successfully manages concurrent sensor polling—monitoring thermal data, luminosity, and security interrupts—in true parallel. Furthermore, the strategic architectural pivot from the physical NodeMCU blueprint to the ESP32 digital twin environment provided incontrovertible evidence of operation. This validation proved that the core logic is highly adaptable, hardware-agnostic, and capable of transforming individual, fragmented components into a cohesive, centralized smart home simulation.

Future Scope & Scalability

While the edge-computing logic of this prototype is fully finalized, the architecture is inherently designed for extensive scalability. Future iterations of this residential ecosystem will focus on the following integrations:

1. **Cloud Integration and Telemetry:** Leveraging the built-in Wi-Fi capabilities of the ESP32/NodeMCU to establish an MQTT (Message Queuing Telemetry Transport) connection. This will allow the microcontrollers to publish real-time environmental data to a global IoT dashboard (such as ThingsBoard or AWS IoT), enabling remote user monitoring from any smart device.
2. **Machine Learning (Predictive Climate):** Upgrading the static thermal thresholds (e.g., 30°C) with lightweight, edge-based Machine Learning (TinyML) models. This would allow the system to predict user comfort preferences based on historical data and seasonal trends, pre-emptively activating the HVAC system before the environment becomes uncomfortably warm.
3. **Custom Printed Circuit Board (PCB) Fabrication:** Transitioning the prototype from a breadboard topology to a finalized, surface-mounted PCB. This will drastically reduce electrical noise, minimize the physical footprint of the core logic engine, and prepare the ecosystem for commercial-grade manufacturing.